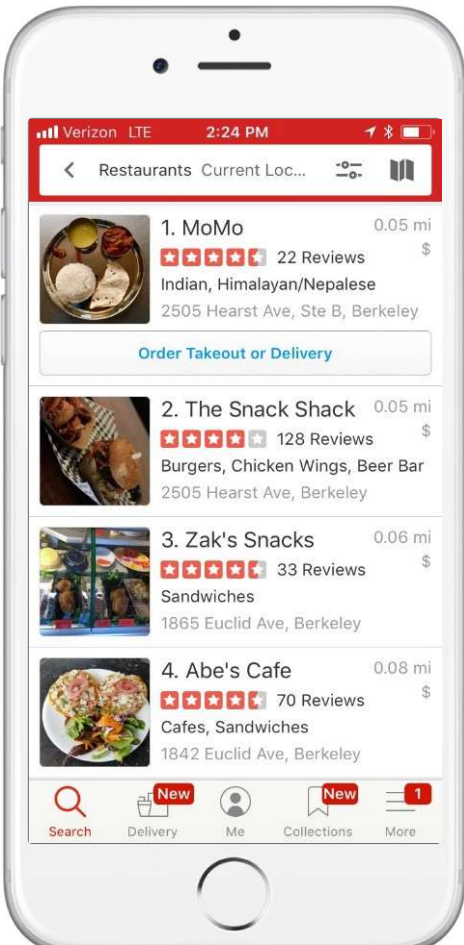


Introduction to Database Systems

Why? DBMS

- Data processing backs essentially every app
- Databases of one form or another back most apps
- Data is at the center of modern society

Where shall I eat, Database?

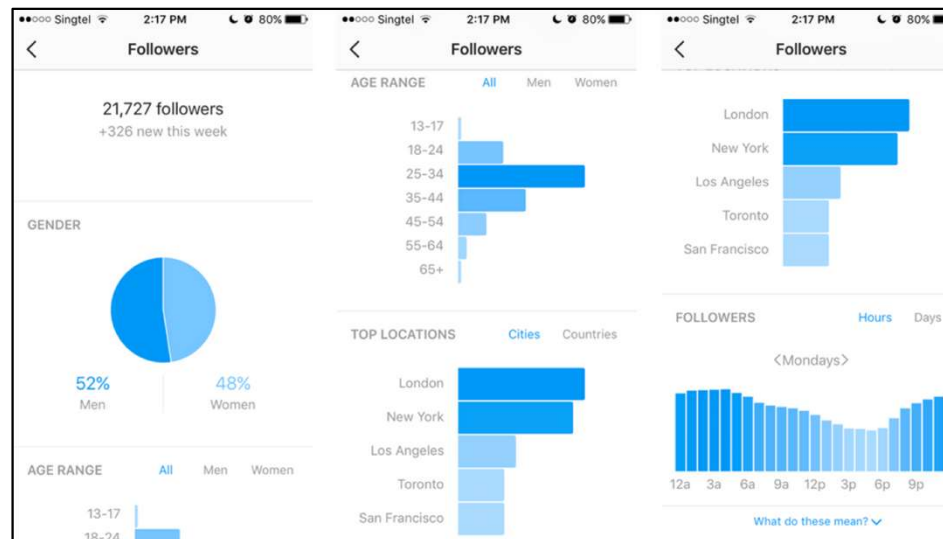
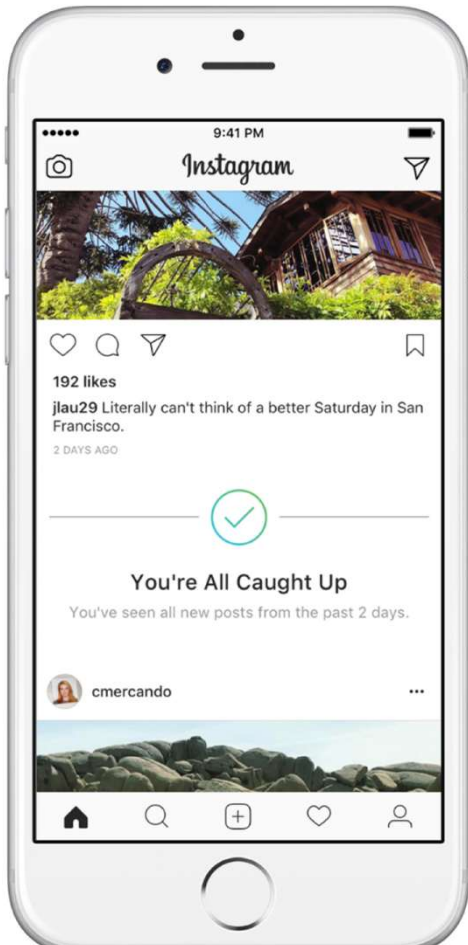


Each ratings star added on a Yelp restaurant review translated to anywhere from a 5% to a 9% effect on revenues.

—Harvard Business School, 2011

<http://hbswk.hbs.edu/item/the-yelp-factor-are-consumer-reviews-good-for-business>

What am I missing, Database?



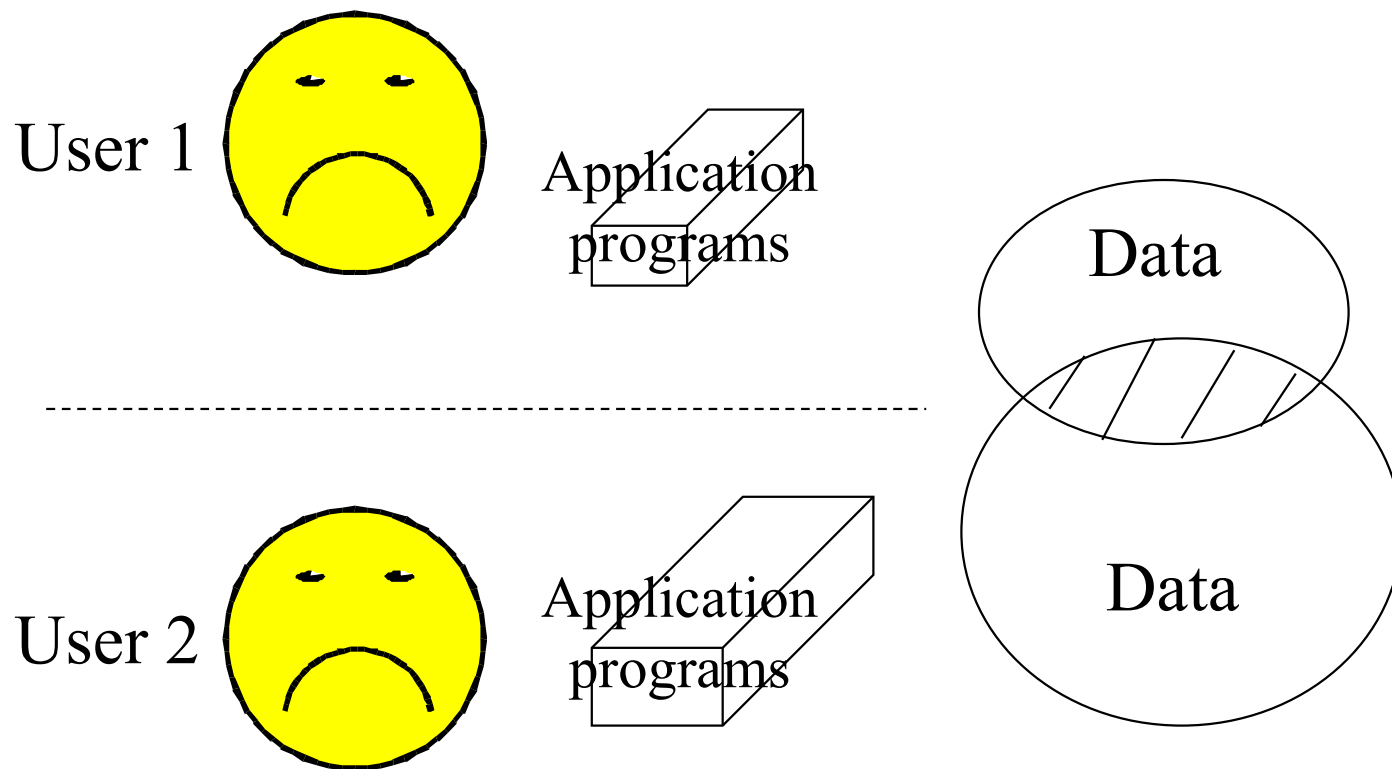
<https://blog.bufferapp.com/instagram-analytics>

<https://instagram-press.com/blog/2018/07/02/introducing-youre-all-caught-up-in-feed/>

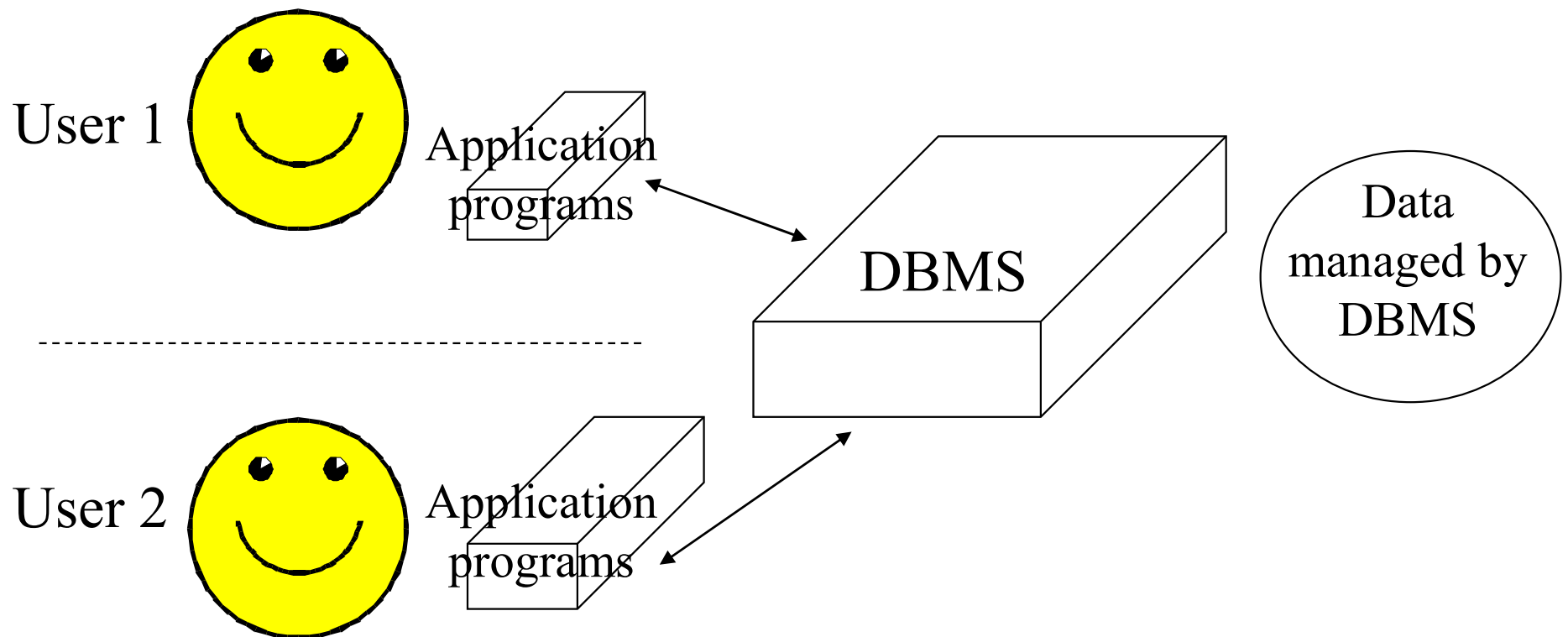
Your career...

- 2000's:
 - Shift from “programs” to apps over data-centric services
- More recently:
 - [End of the full-stack programmer](#)
 - New, ubiquitous professions:
 - Data Scientist, Data Engineer, Machine Learning Engineer
 - Evolution of IT: people used to be in charge of data center and its operation, now operations are handled in the cloud and data management is left for IT professionals

BEFORE DBMS



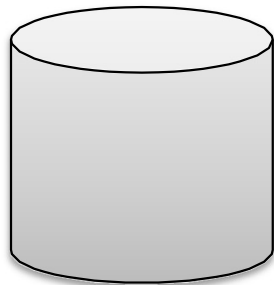
AFTER DBMS



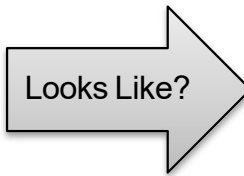
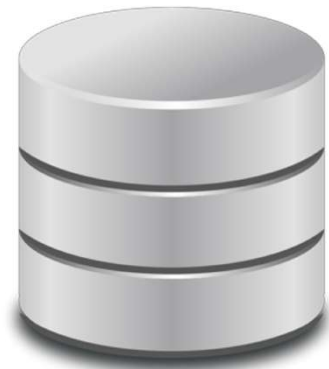


Increasingly many companies see themselves as data driven.

Universal Symbol for a Database



Why the Symbol?



Platters on a Disk Drive



Is This a Database?

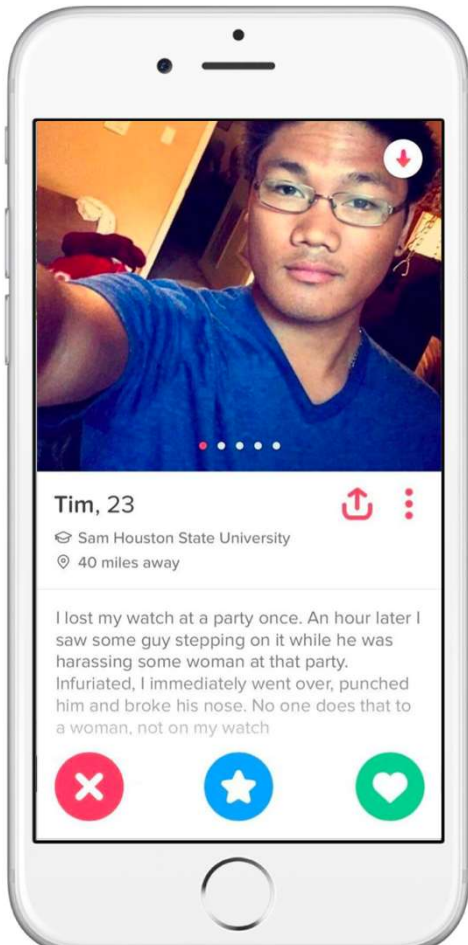
- Rolodex
- Alphabetically ordered cards
- Indexed access by first letter





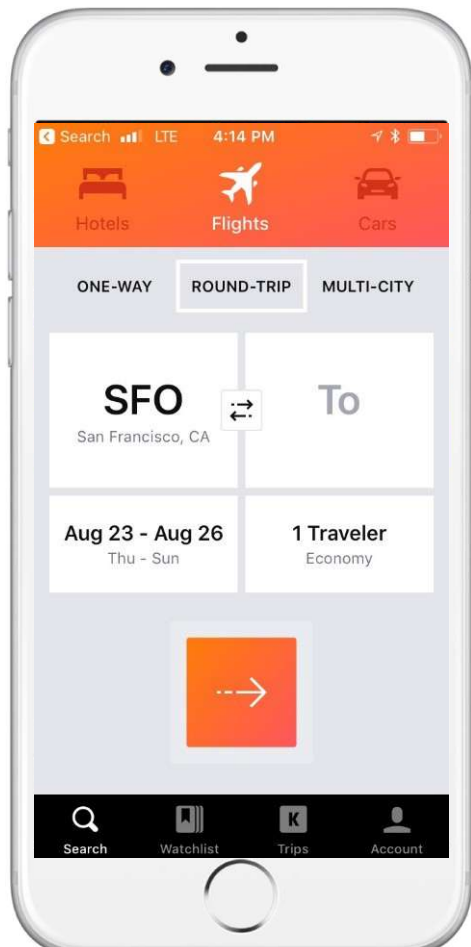
Is This a Database?

- A database + “business logic” + user interface?



Is This a Database?

- Airline reservation systems were one of the earliest pervasive consumer uses of database systems.
 - IBM/American Airlines' SABRE system, 1964.
 - “Semi-Automated Business Research Environment”
 - Travelocity.com a direct descendant of SABRE
 - Acquired by Expedia, 1/2015



DATABASE & DBMS

- **Database:** An integrated collection of data, usually stored on secondary storage, typically describing the activities of one or more related organizations.

For Example, a university database might contain information about:

- Entities such as students, faculty, courses, and classroom.
 - Relationships between entities: students enrolled in courses, faculty teaching courses, courses using rooms.
- **Database management system (DBMS):** A collection of software/programs designed to assist in maintaining and utilizing large collections of data.

Relational DBMSs

- Traditionally DBMS referred to relational databases



- RDBMS** is a more appropriate term
- SQL** data description and manipulation language
- ACID** transaction consistency: How the data remains consistent
- Durable** writes (prevent data loss)
- Mature** technologies ...

343 systems in ranking, January 2019

Rank			DBMS	Database Model	Score		
Jan 2019	Dec 2018	Jan 2018			Jan 2019	Dec 2018	Jan 2018
1.	1.	1.	Oracle +	Relational DBMS	1268.84	-14.39	-73.11
2.	2.	2.	MySQL +	Relational DBMS	1154.27	-6.98	-145.44
3.	3.	3.	Microsoft SQL Server +	Relational DBMS	1040.26	-0.08	-107.81
4.	4.	4.	PostgreSQL +	Relational DBMS	466.11	+5.48	+79.93
5.	5.	5.	MongoDB +	Document store	387.18	+8.57	+56.24
6.	6.	6.	IBM Db2 +	Relational DBMS	179.85	-0.90	-10.43
7.	7.	↑ 9.	Redis +	Key-value store	149.01	+2.19	+25.88
8.	8.	↑ 10.	Elasticsearch +	Search engine	143.44	-1.26	+20.89
9.	9.	↓ 7.	Microsoft Access	Relational DBMS	141.62	+2.10	+14.92
10.	10.	↑ 11.	SQLite +	Relational DBMS	126.80	+3.78	+12.54

Based on #mentions (e.g., stack overflow), google trends, job postings, profile data on LinkedIn, tweets ...

<http://db-engines.com/en/ranking>

Market Trends

- Cloud DBMS disrupting on-premises vendors
 - Cloud is less relational-centric
 - But fastest-growing services at AWS are RDBMSs
- “One size doesn’t fit all”
 - Main-memory DBMS
 - Graph DBMS
 - TimeSeries DBMS
 - Key-Value Stores (NoSQL)
 - Analytics Platforms (Spark, Hadoop)

WHY A DBMS?

1. Reduced application development time
2. Data independence: Application programs not dependent on data representation and storage details
3. Data sharing: data is better utilized (discovered and reused), redundancy of data is minimized
4. Data integrity and consistency: one may enforce consistency constraints on data, e.g., number of seats sold $\leq$ number of seats on the plane $\times 1.1$
5. Centralized control: DBA tunes the database to balance user's needs
6. Security: mechanisms to prevent unauthorized access. These mechanisms are based on content instead of file-oriented approach.
7. Concurrency control: avoids undesirable race conditions that arise with simultaneous access/updates to data
8. Crash recovery: ensures the integrity of data in the presence of failures

From a user's perspective

1. **Foundations:** Relational data models & SQL
 - How to manipulate data with SQL, a declarative language
 - *reduced expressive power but the system can do more for you*
2. **Database Design:** Design theory and constraints
 - Designing relational schema to keep your data from getting corrupted

Database internals

3. Introduction to database systems

- Data Storage and IO models
- File Organization

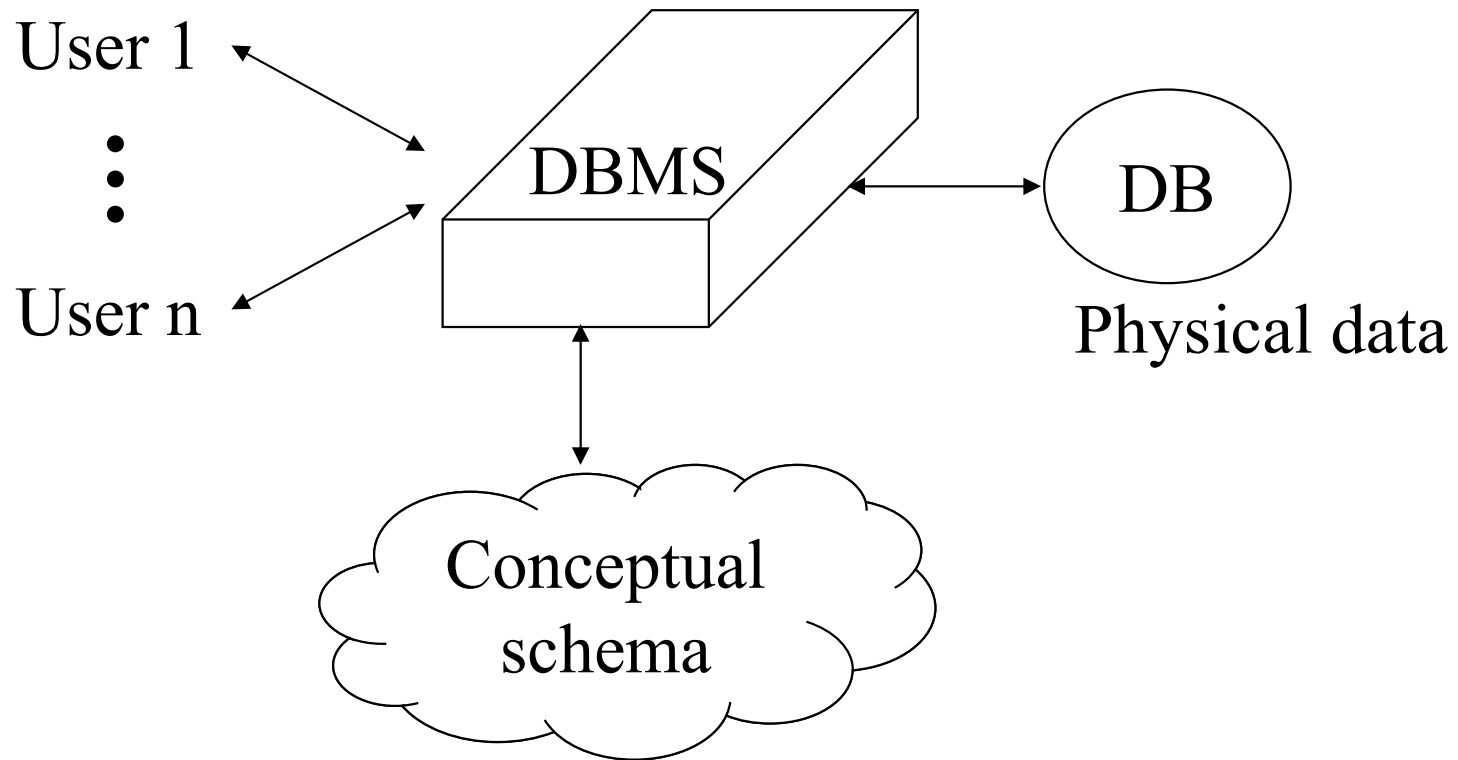
4. Indexing and Hashing

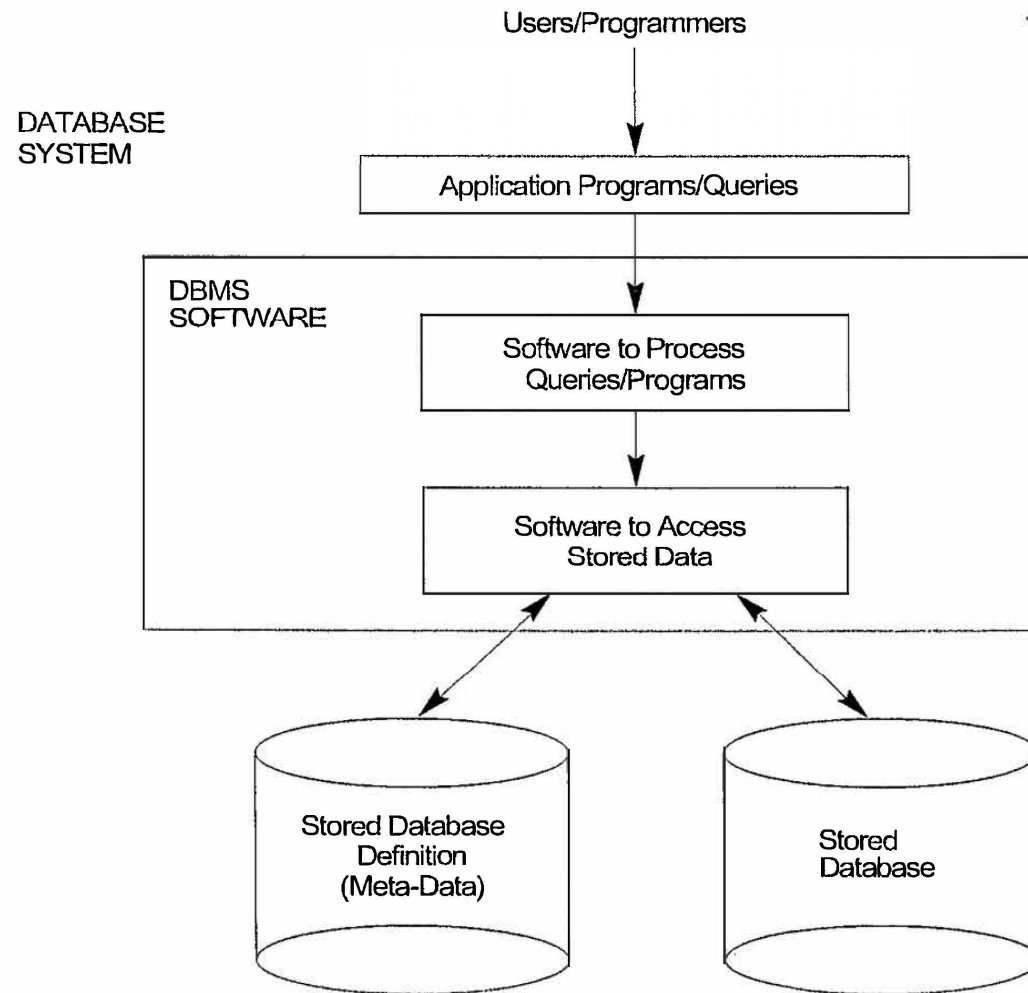
- Intro to indexing
- B+ Tree, Hash Indexes

5. Query processing

- Access methods and operators
- Joins
- Relational algebra and Query optimization

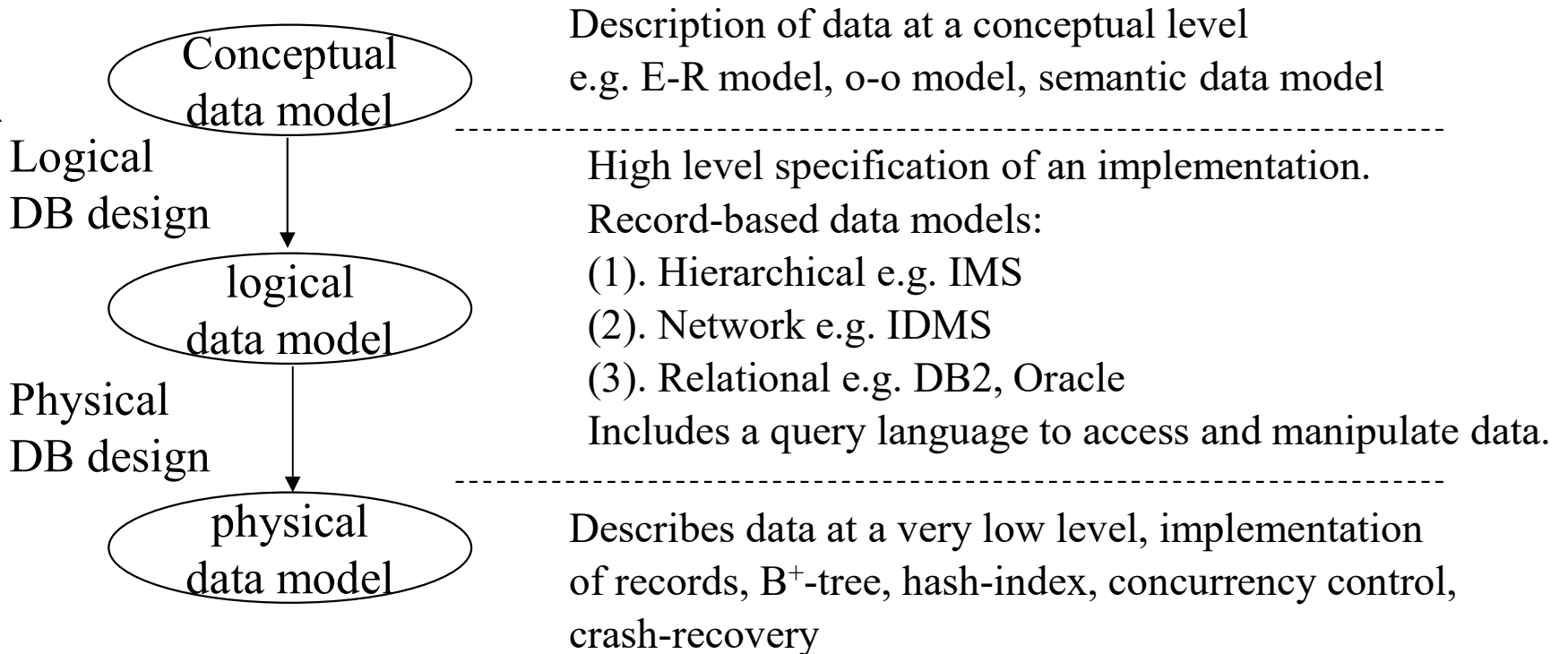
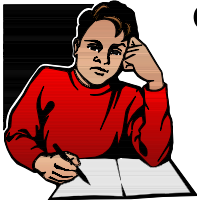
DATABASE MANAGEMENT SYSTEMS ARCHITECTURE

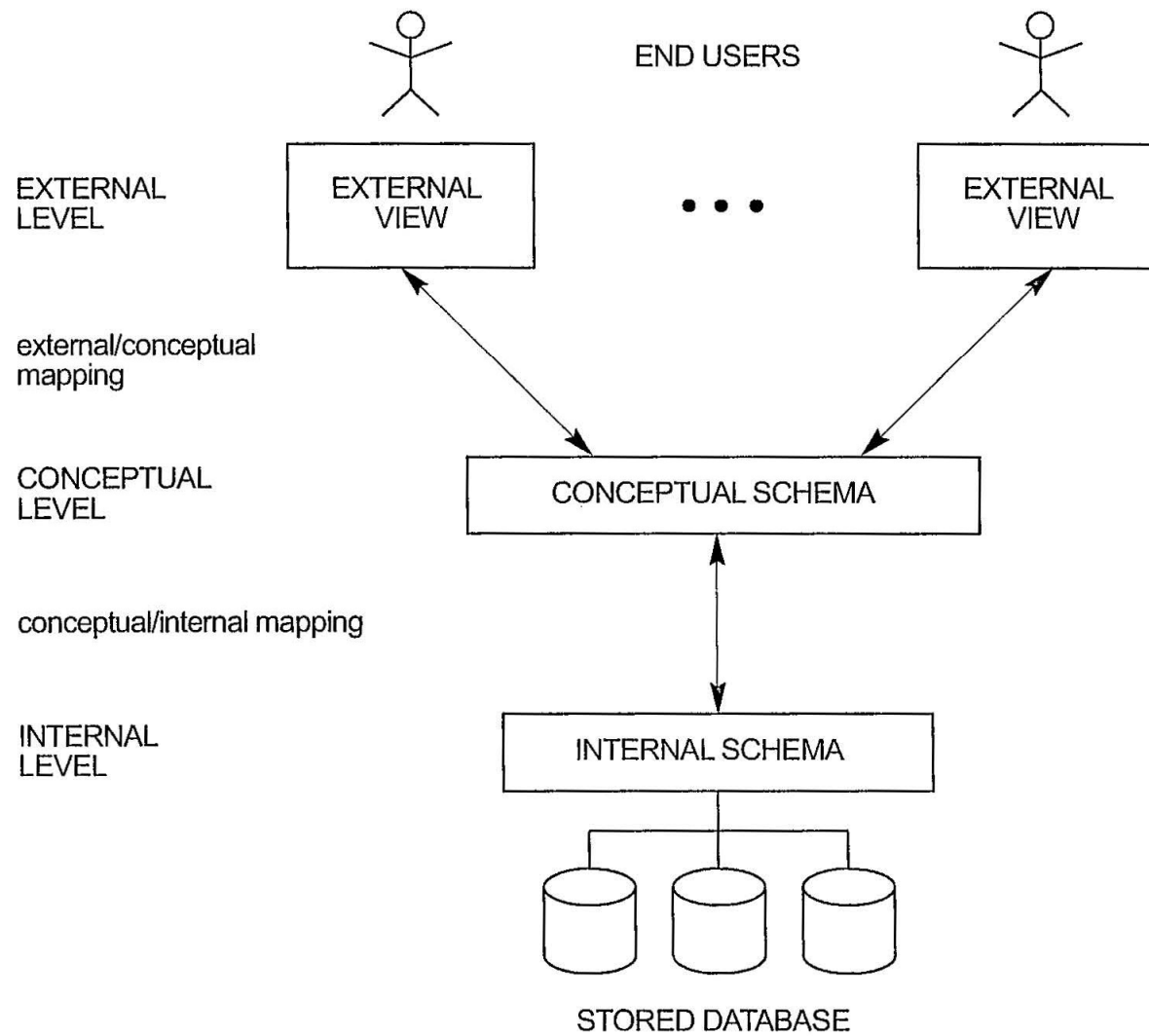




DATA MODEL

- Underlying the structure of a database is a data model
- **Data model**: a specification tool (class of data structures) for describing data, including entities, relationships, semantics, constraints, etc. Purpose: to capture information about real world data.





DATA INDEPENDENCE

1. Physical data independence: modify the physical scheme (data structures, e.g., B-tree or hash index) without causing application programs to be rewritten. These modifications are necessary to enhance performance and new software releases. Most relational vendors support this kind of data independence.
2. Logical data independence: Modify the conceptual scheme (e.g., add a new attribute to a table, rename an attribute) without causing application programs to be rewritten. This kind of data independence is harder to achieve.

DATABASE LANGUAGES

There are several languages associated with a database:

1. [Data Definition Language \(DDL\)](#): The database scheme is specified by a set of definitions that are expressed by a special language named DDL. The result of compiling DDL statements is a set of tables stored in a file called data dictionary. This file contains meta-data (data about the data stored in the database).
2. [Data Manipulation Language \(DML\)](#): a language that enables users to access or manipulate data (retrieve, insert, replace, delete) as organized by a certain data model. We will look at a commercial DML named SQL. In general, there are two types of DML:
 - Procedural: Describes what data is needed and how to get it: e.g., relational algebra
 - Non-procedural: Describes what data is needed without specifying how to get it: e.g., tuple relational calculus

SYSTEM USERS

There are several kind of users associated with a system:

- Database administrator: defines schemas, storage structures and access method definitions, physical organization, authorization, integrity constraints.
- Application programmers: they write a program and make it available to the end-users
- Sophisticated users: they use a query language (SQL) to access the database interactively
- Naive (end) users: they invoke the application programs

E-R CONCEPTUAL DATA MODEL

E-R MODEL

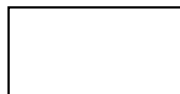
- E-R model is based on a real world that consists of entities and relationships. It consists of two primitives:
 1. **Entities**: objects that exist and are distinguishable from one another. These objects might be abstract, e.g., SF-49ers, Joe, Bob, etc.
 2. **Relationships**: two or more entities may participate in a relationship, e.g., Joe is a fan of SF-49ers, etc.
- An entity is represented by a set of *attributes*, e.g., Joe might have a social-security number, a phone number, a spouse, and a salary
- An *attribute* is a function that maps an entity into a domain, e.g., Joe's social-security number maps Joe into the domain of integers. An attribute is represented as an

single

MV

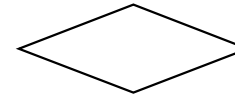
Derived

- A set of entities of the same type is termed an *entity set*, e.g., football teams. Entity sets do not need to be disjoint, e.g., students, employees, people. An entity set is represented as a

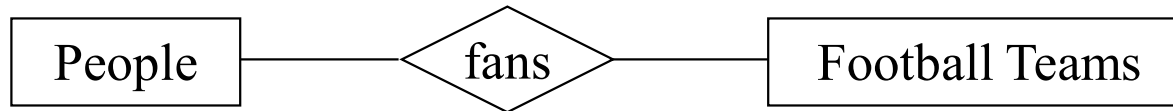


RELATIONSHIPS

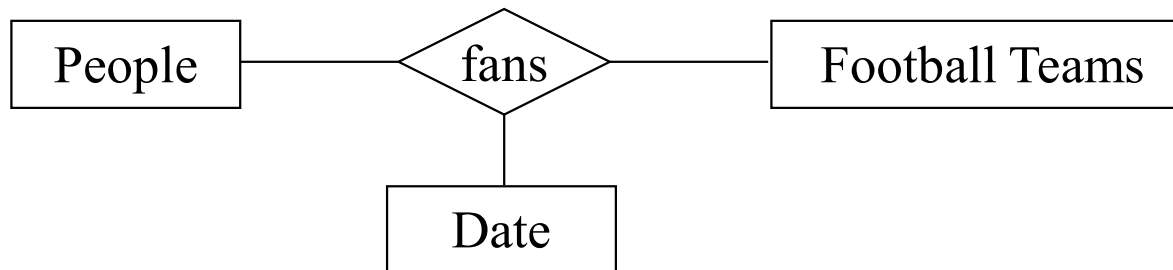
- Relationships of the same type are contained in a relationship set. More formally, if $E_1, E_2, \dots, E_n$ are entity sets, then a relationship set R is a subset of: $\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$ where $(e_1, e_2, \dots, e_n)$ is a relationship. A relationship set is represented as a



- A relationship may involve two entities, binary relationship:

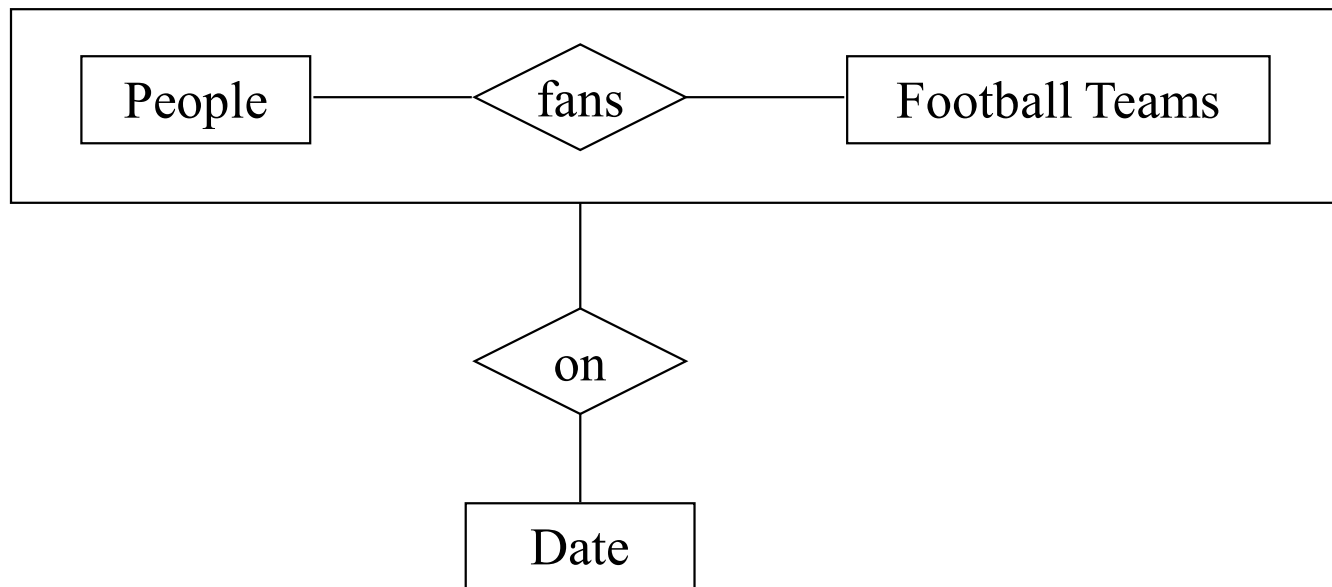


- A relationship may involve three entities, ternary relationship:



RELATIONSHIPS (Cont...)

- A relationship may involve n entities, N-ary relationship
- It is always possible to replace a non-binary relationship set by a number of distinct binary relationship sets



RELATIONSHIPS (Cont...)

- Example: A library database contains a listing of authors that have written books on various subjects (one author per book). It also contains information about libraries that carry books on various subjects.

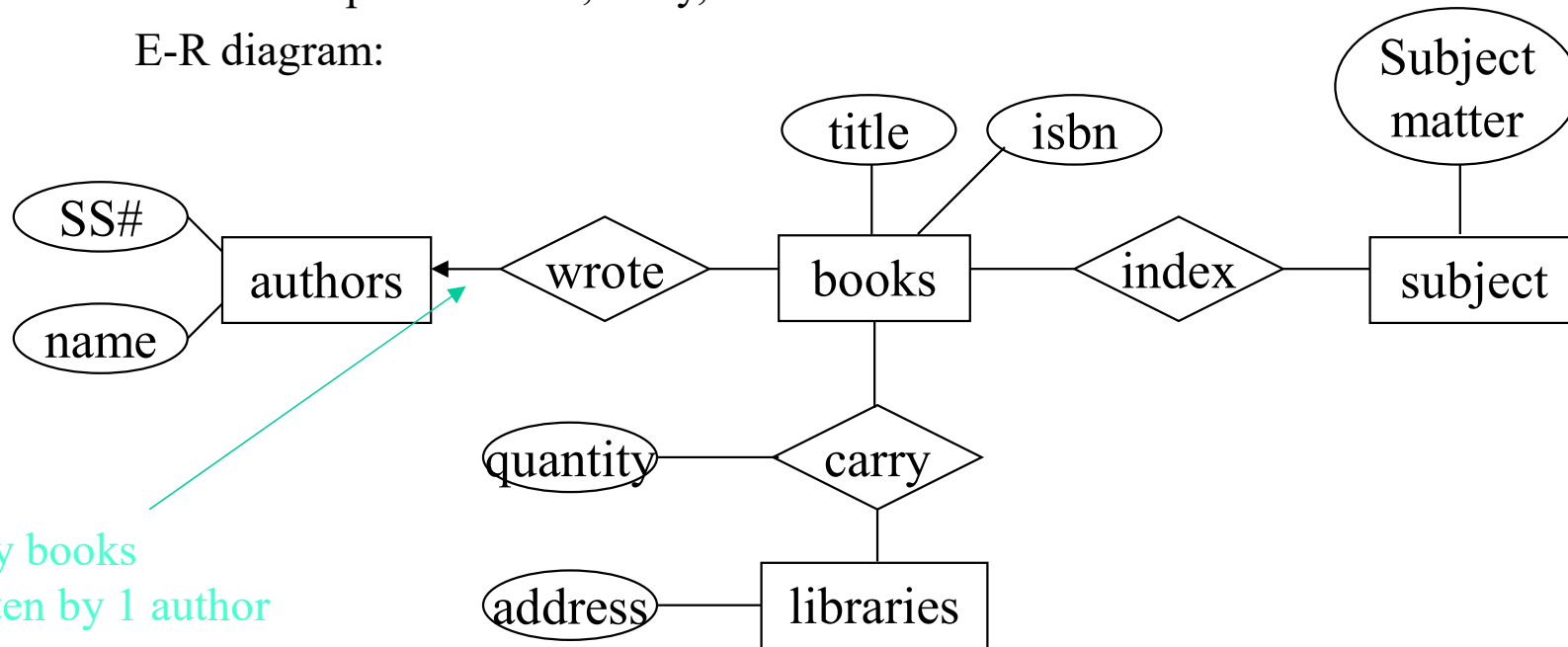
RELATIONSHIPS (Cont...)

- Example: A library database contains a listing of authors that have written books on various subjects (one author per book). It also contains information about libraries that carry books on various subjects.

Entity sets: authors, subjects, books, libraries

Relationship sets: wrote, carry, indexed

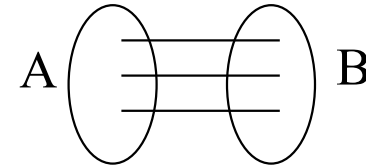
E-R diagram:



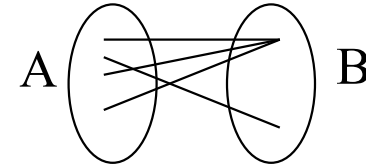
BINARY RELATIONSHIP

A binary relationship between entity set A and B might be:

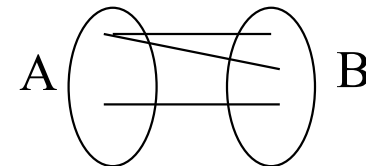
- 1:1 Women marrying Men (function)



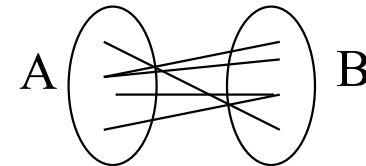
- N:1 Children having mothers (function)



- 1:N Mothers having children (inverse function)



- M:N Students enrolled in a class

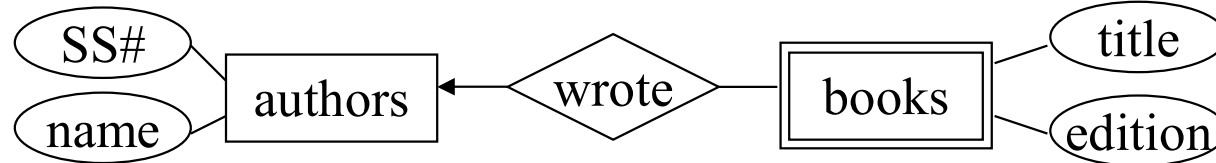


KEY

- Entities and relationships are distinguishable using various keys
- A *key* is a combination of one or more attributes, e.g., social-security number, combination of name and social-security number.
- A *superkey* is a key defined either for an entity set or relationship set that uniquely identifies an entity, e.g., social-security number, phone number, combination of name and social-security number.
- A *candidate key* is a minimal superkey that uniquely identifies either an entity or a relationship, e.g., social-security number, phone number.
- A *primary key* is a candidate key that is chosen by the database designer to identify the entities of an entity set.

KEY (Cont...)

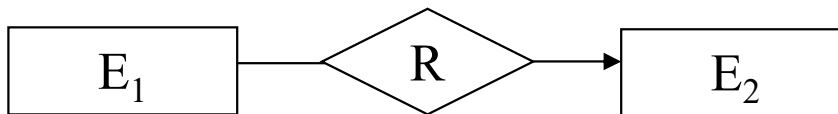
- A **foreign key** is a set of one or more attributes of a strong entity set that are employed to construct the discriminator of a weak entity set. The primary key of a weak entity set is formed by the primary key of the strong entity set on which it is existence-dependent.



- Relationship sets also have primary keys. Assume R is a relationship set involving entity sets $E_1, E_2, \dots, E_n$. Let $\text{primary-key}(E_i)$ denote the primary key for entity set E_i . Assume $\text{primary-key}(E_i)$ is unique for $1 \leq i \leq n$. If R has no attributes then its superkey is:

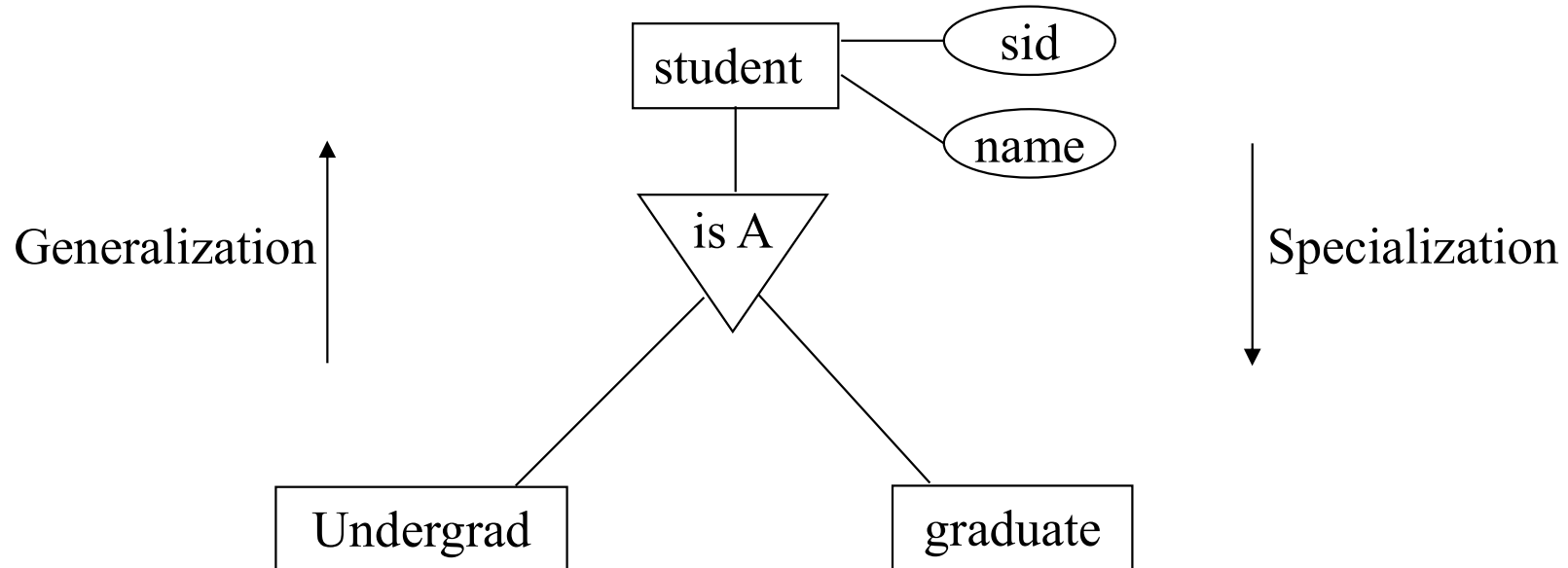
$$\text{primary-key}(E_1) \cup \text{primary-key}(E_2) \cup \dots \cup \text{primary-key}(E_n)$$

- This is a primary key if the mapping constraint is many-to-many.
- If the mapping constraint is many to one from E_1 to E_2 then the primary key of R is primary key of E_1 .



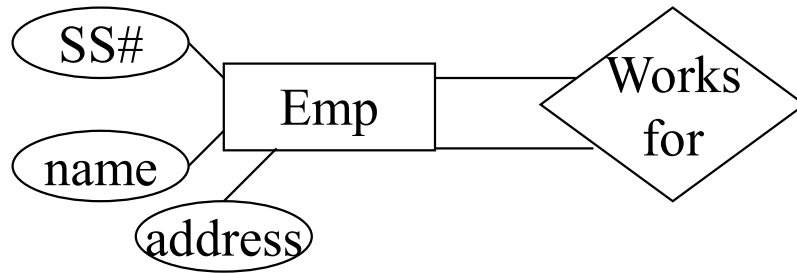
GENERALIZATION AND SPECIALIZATION

- **Generalization** is the result of computing the union of two or more entity sets to produce a higher-level entity set. It represents the containment relationship that exists between the higher-level entity set and one or more lower-level entity sets.
- **Specialization** constructs the lower level entity sets that are a subset of a higher level entity set.



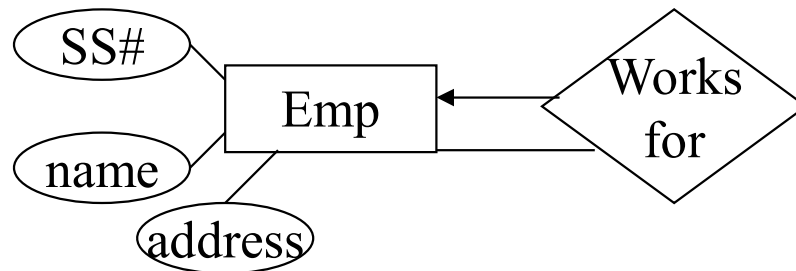
EXAMPLE

- Employees of a large company, e.g., IBM, where an employee reports to a manager. The manager is also an employee who reports to another manager. This chain of command continues to the very top where the CEO is the only employee who is not reporting to a manager. Draw the ER diagram for this example.



Emp(SS#, name, address)

WorksFor(SS#, mgrSS#)



Emp(SS#, name, address, mgrSS#)

RELATIONAL DATA MODEL

BASIC CONCEPTS

- Relational data model consists of a collection of tables, each with a unique name. A table is termed a *relation*.
- A **row** (also termed a **tuple** or a **record**) in table R represents a relationship among a set of values. Each value corresponds to an attribute or a field. A column of a table represents an attribute. Given a relation with attributes $(A_1:D_1, A_2:D_2, \dots, A_n:D_n)$ the domain of this relation is $D_1 \times D_2 \times \dots \times D_n$. However, an instance of a relation is a subset of this domain (recall “value & a variable”), e.g., COMP282 (stdid:integer, name:string, gpa:float)

stdid	name	gpa
101	Adams	3.4
102	Jones	2.9
103	Hayes	3.5
104	Smith	3.1

- A *tuple variable* t refers to a tuple in the relation. One may access the value of an attribute contained in a tuple variable by $t[\text{attr-name}]$. e.g., $t[\text{stdid}]$

$t[\text{stdid}] = 101$ $t[\text{name}] = \text{“Hayes”}$ $t[1] = 102$

- Database *scheme* is the data about the data. It describes the logical design of the database.

student-schema = (stdid , name , gpa) schema for relation student

COMP282(student-schema) COMP282 is a relation on student-schema

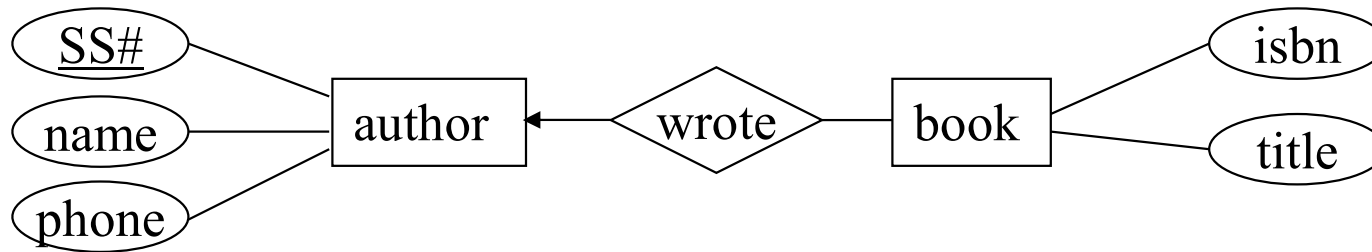
- Database *instance* is the data in the database at a given instant in time.
- Entity and relationship sets are represented by relations

TABLES GENERATION

- Transforming an entity set to a table (relation):
 1. Strong entity set with attributes $a_1, a_2, \dots, a_n$: represent it as a table with n unique columns (one column per attribute).
 2. Weak entity set with attributes $a_1, a_2, \dots, a_n$ and a foreign key $b_1, b_2, \dots, b_m$: represent it as a table with $n+m$ columns, one for each of $\{a_1, a_2, \dots, a_n\} \cup \{b_1, b_2, \dots, b_m\}$.
- Transforming relationship sets to tables:
 1. Relationship set involving entity sets $E_1, E_2, \dots, E_n$, each entity having primary key $\text{prim-key}(E_i)$, represent it as a relation with each attribute corresponding to
 - $\bigcup_{i=1}^n \text{Prim_key}(E_i)$
 2. if the relationship has attributes $a_1, a_2, \dots, a_n$ then the relation consists of:
 - $\{a_1, a_2, \dots, a_n\} \cup \bigcup_{i=1}^n \text{Prim_key}(E_i)$

TABLES GENERATION (Cont...)

3. For a functional relationship (1:1, M:1, 1:M) from set A to set B, incorporate the primary key of B into A



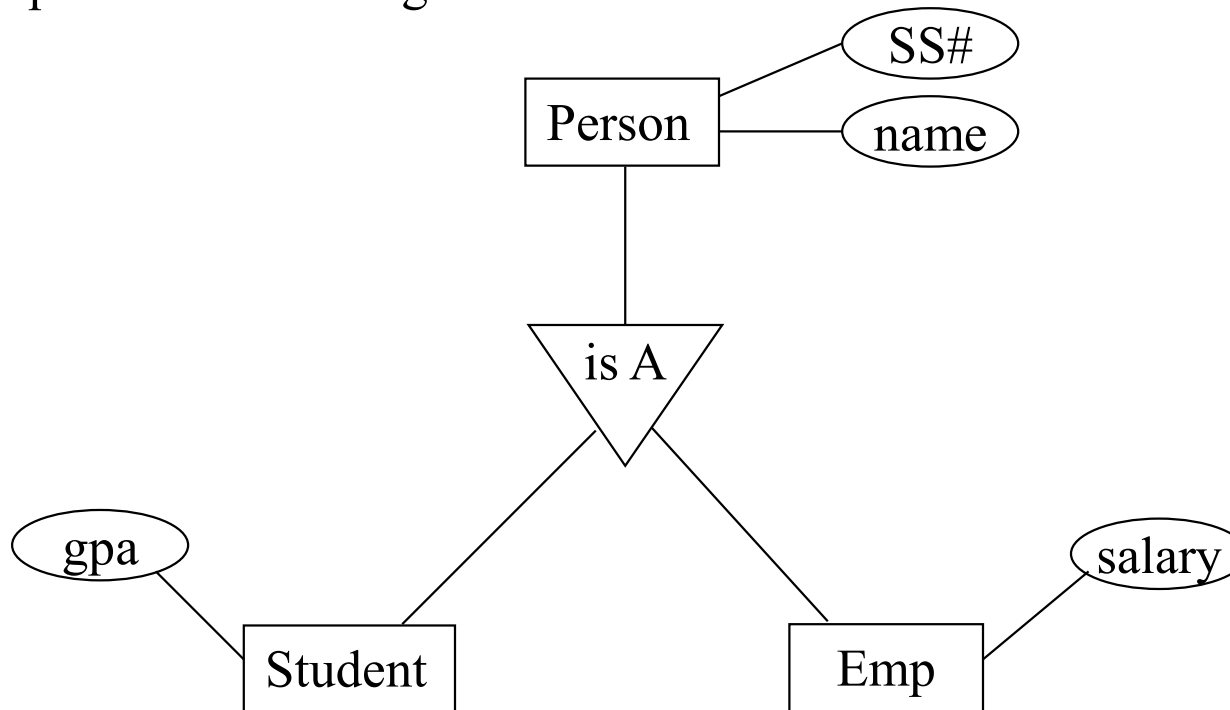
Author(SS#, name, phone)

Book(SS#, isbn, title)

- The number of attributes in a relation R is termed *arity* or *degree* of relation R.
- The number of tuples in a relation R is termed the *cardinality* of R.

TABLES GENERATION (Cont...)

- For specialization and generalization:



TABLES GENERATION (Cont...)

Two ways to generate tables:

1. Approach 1:

- A table for the higher level entity set
- For each lower entity set, create a table that includes a column for each of the attributes of that entity set plus a column for each attribute of the primary key of the higher-level entity set

Person(SS#, name)

Student(SS#, gpa)

Emp(SS#, salary)

2. Approach 2:

- No table for the higher level entity set
- One table for each lower entity set with the following columns:
 - One column for each local attribute
 - One column for each attribute of the higher level set

Student(SS#, name, gpa)

Emp(SS#, name, salary)

A Motivating, Running Example

- Consider building a course management system (CMS):

- Students
- Courses
- Professors



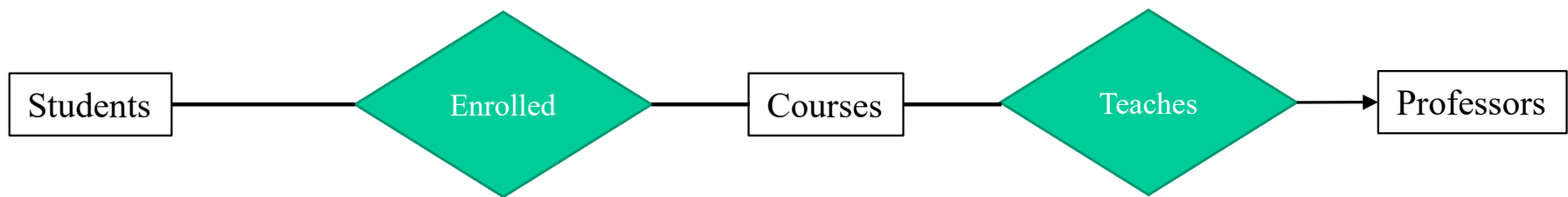
Entities

- Who takes what
- Who teaches what



Relationships

E-R Model (CMS)



Modeling the Course Management System

- *Logical Schema*
 - Students(sid: *string*, name: *string*, gpa: *float*)
 - Professors(pid: *string*, name: *string*)
 - Courses(cid: *string*, cname: *string*, section: *int*, semester: *string* credits: *int*, pid: *string*)
 - Enrolled(sid: *string*, cid: *string*, grade: *string*)

Course Management System cont

Student	sid	Name	Gpa
	101	Bob	3.2
	123	Jane	3.8

Professors	pid	Name
	10111	Bahram
	12311	Jane

Courses	cid	cname	section	semester	crdits	pid
	182	DS	11832	sp25	3	10111
	282	Advanced DS	11500	su25	3	12311

Enrolled	sid	cid	Grade
	101	182	A
	123	282	A
	101	282	A

Thursday, August
6, 2026

Data independence

Concept: Applications do not need to worry about *how the data is structured and stored*

Logical data independence:

protection from changes in the *logical structure of the data*

*I.e., should not need to ask:
can we add a new entity or
attribute without rewriting
the application?*

Physical data independence:

protection from *physical layout changes*

*I.e. should not need to ask:
which disks are the data
stored on? Is the data
indexed?*

One of the most important reasons to use a DBMS

Example of a Traditional Database Application

Suppose we are building a system to store the information about:

- students
- courses
- professors
- who takes what, who teaches what

Functionality of a DBMS

The programmer sees SQL, which has two components:

- Data Definition Language – DDL
 - Define and modify schema
- Data Manipulation Language - DML
 - Query language

Behind the scenes the DBMS has:

- Query engine
- Query optimizer
- Storage management
- Transaction Management (concurrency, recovery)

How the Programmer Sees the DBMS

- Start with DDL to *create tables*:

```
CREATE TABLE Students (  
    Name CHAR(30)  
    SSN CHAR(9) PRIMARY KEY NOT NULL,  
    Category CHAR(20)  
) ...
```

- Continue with DML to *populate tables*:

```
INSERT INTO Students  
VALUES('Charles', '123456789', 'undergraduate')  
... ..
```

How the Programmer Sees the DBMS

- Tables:

Students:

SSN	Name	Category
123-45-6789	Charles	undergrad
234-56-7890	Dan	grad
	...	...

Takes:

SSN	CID
123-45-6789	CSE444
123-45-6789	CSE444
234-56-7890	CSE142
	...

Courses:

CID	Name	Quarter
CSE444	Databases	fall
CSE541	Operating systems	winter

- Still implemented as files, but behind the scenes can be quite complex

“data independence” = separate logical view from physical implementation

Queries

- Find all courses that “Mary” takes

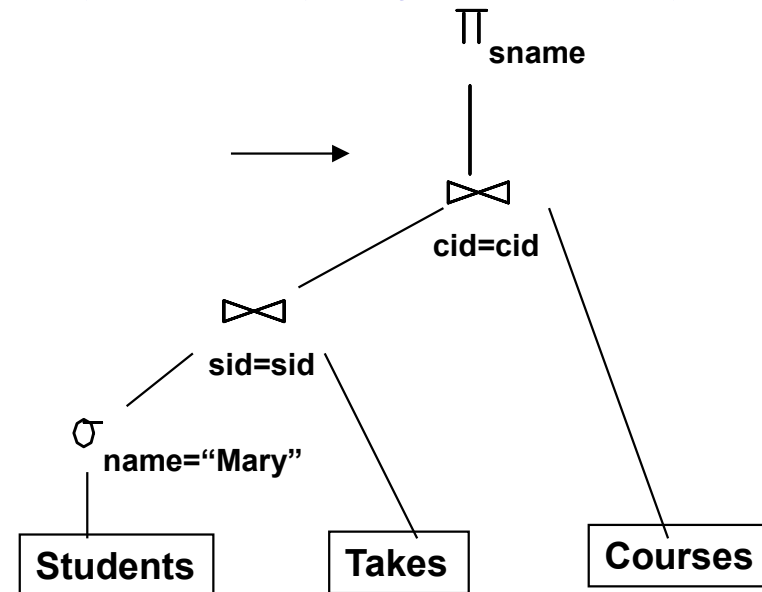
```
SELECT C.name
FROM   Students S, Takes T, Courses C
WHERE  S.name="Mary" and
       S.ssn = T.ssn and T.cid = C.cid
```

- What happens behind the scenes?
 - Query processor figures out how to answer the query efficiently.

Queries, behind the scene

Declarative SQL query → *Imperative query execution plan:*

```
SELECT C.name
FROM Students S, Takes T, Courses C
WHERE S.name="Mary" and
      S.ssn = T.ssn and T.cid = C.cid
```



The **optimizer** chooses the best execution plan for a query